DO NOW — RECALL

Lesson 066 · Problem Decomposition: Build a Useful Solu

Name: _____ Class: _____ Date: _____

1. What makes a problem suitable for computation?

2. What is an abstraction?

3. What is a procedure?

4. Stack initially empty: Push A, B, C; Pop; Push D. Give the stack top-to-bottom.

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

Explicit inputs, constraints and a testable outcome.

2. Model answer

A representation keeping relevant detail and omitting irrelevant detail.

3. Model answer

Named subroutine performing a task.

4. Model answer

D, B, A. C is popped and D is pushed above B.

By the end, you can…

• Decomposition divides a larger problem into manageable sub-problems whose inputs and outputs are clear.

• A top-down design begins with a main task, then refines it into independently testable functions or procedures.

• Interfaces between subproblems matter: validation should happen before data are stored or a fee is charged.

Vocabulary — know these terms

decomposition — Split task into smaller problems.

top-down design — Start broad, refine into subtasks.

interface — Inputs/outputs between components.

modularity — Small cohesive program components.

NEW LEARNING

Read, annotate and explain

Define and predict

Decomposition divides a larger problem into manageable sub-problems whose inputs and outputs are clear.

Explain

A top-down design begins with a main task, then refines it into independently testable functions or procedures.

Evaluate

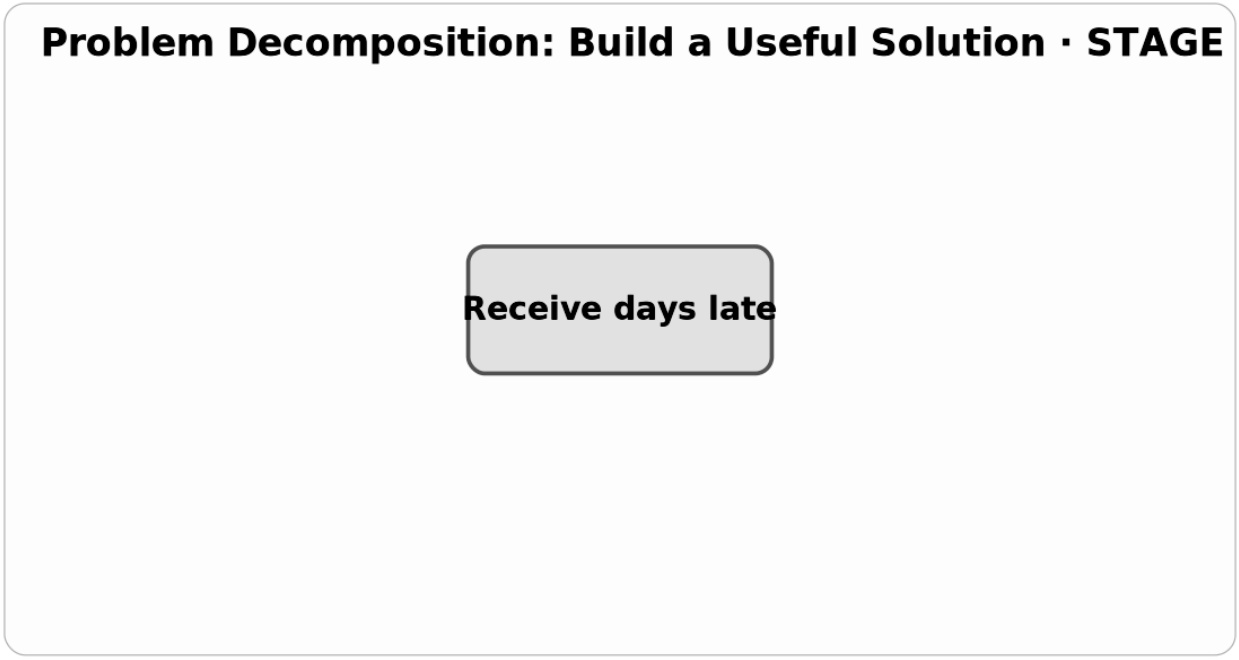
Interfaces between subproblems matter: validation should happen before data are stored or a fee is charged.

Explain one step in your own words, using the correct technical terms.

New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

New Learning

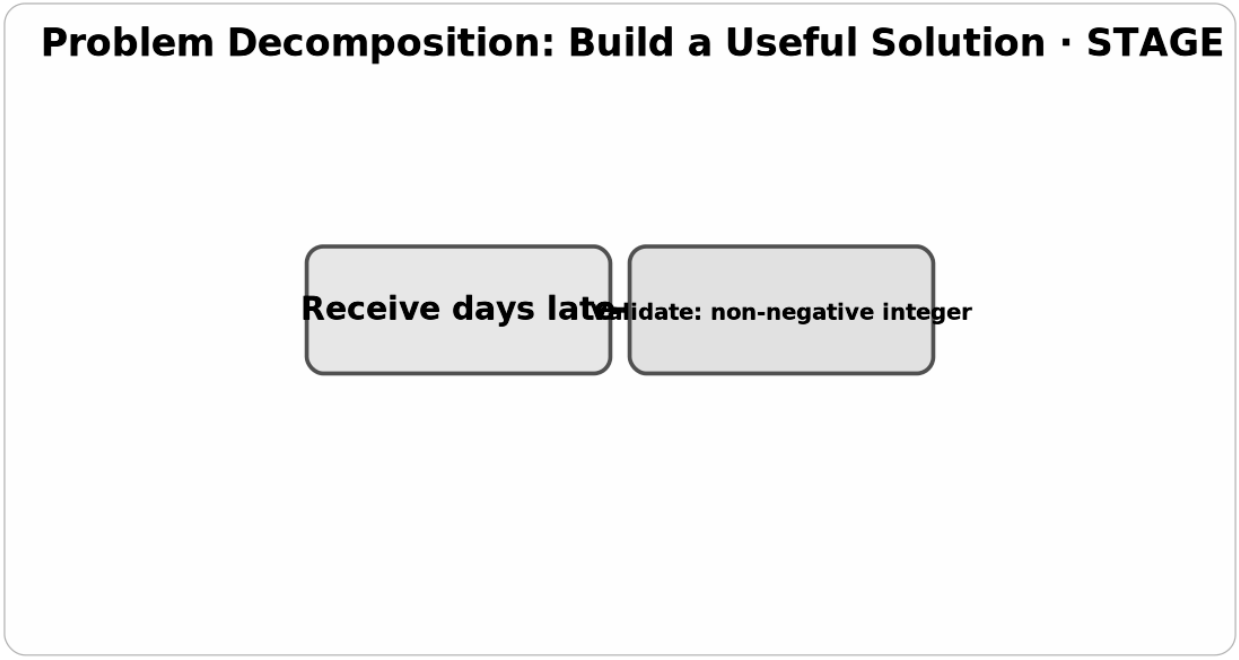


Problem Decomposition: Build a Useful Solution · STAGE 1; Receive days late

New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

New Learning

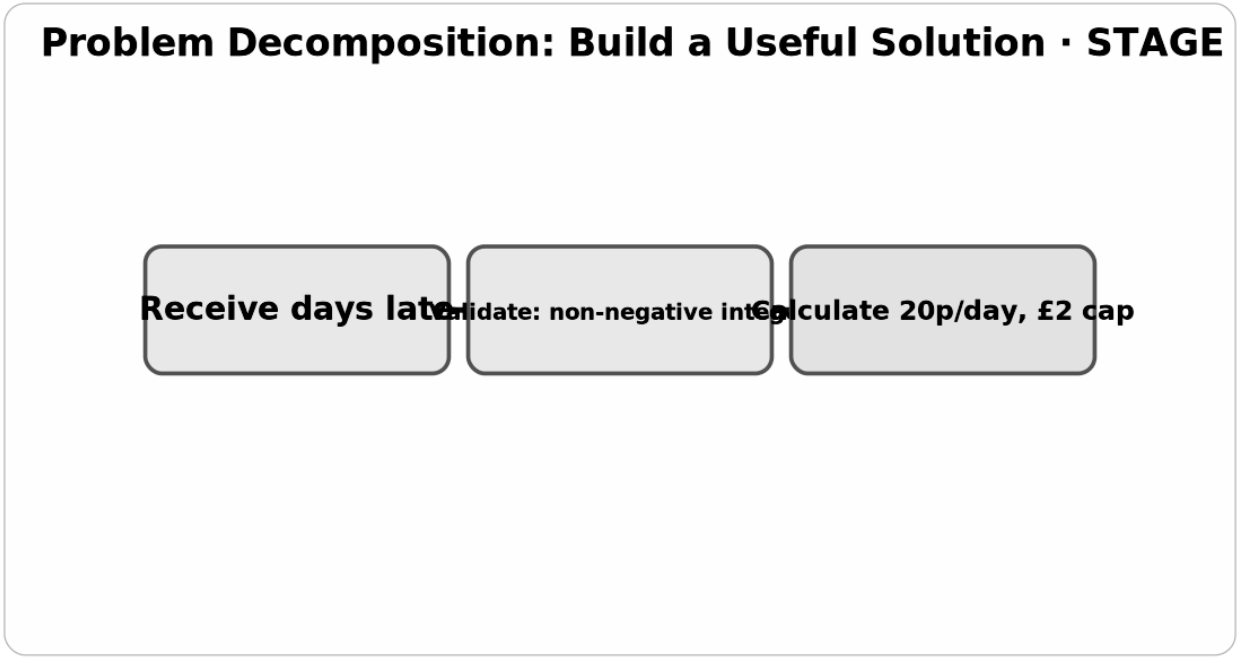


Problem Decomposition: Build a Useful Solution · STAGE 2; Receive days late; Validate: non-negative integer

New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

New Learning

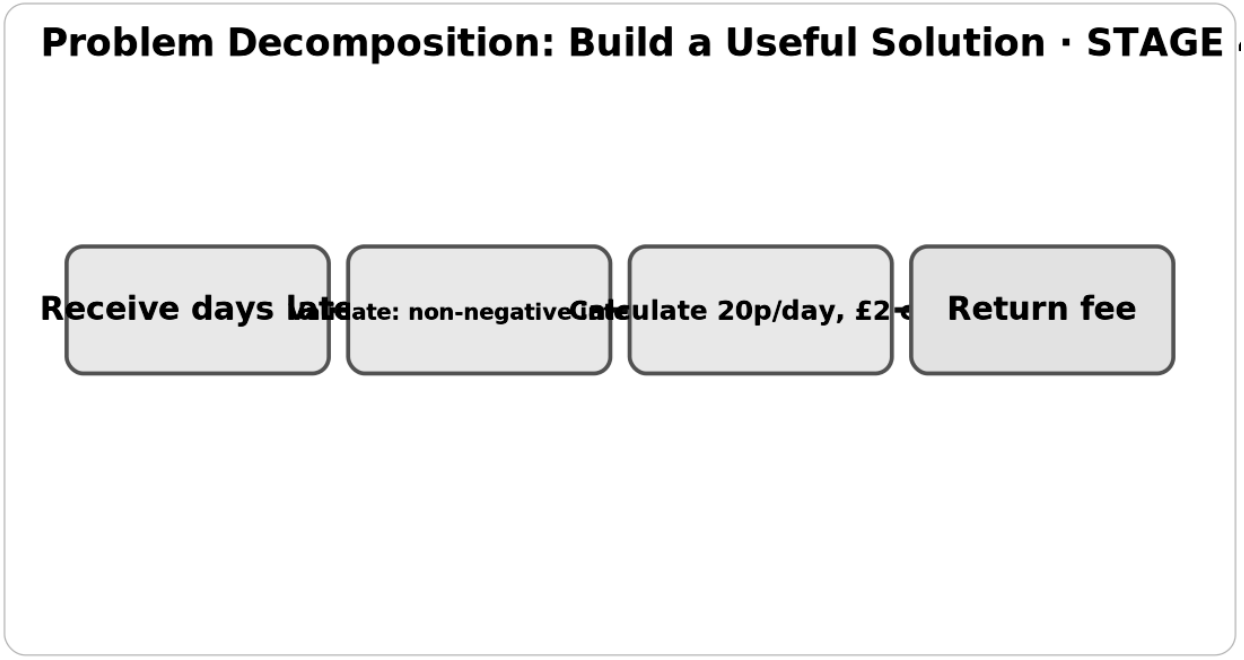


Problem Decomposition: Build a Useful Solution · STAGE 3; Receive days late; Validate: non-negative integer; Calculate 20p/day, £2 cap

New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



Problem Decomposition: Build a Useful Solution · STAGE 4; Receive days late; Validate: non-negative integer; Calculate 20p/day, £2 cap; Return fee

I DO — WORKED EXAMPLE

Follow the worked method

Decompose a cinema price calculator for child/adult tickets.

Follow the reasoning

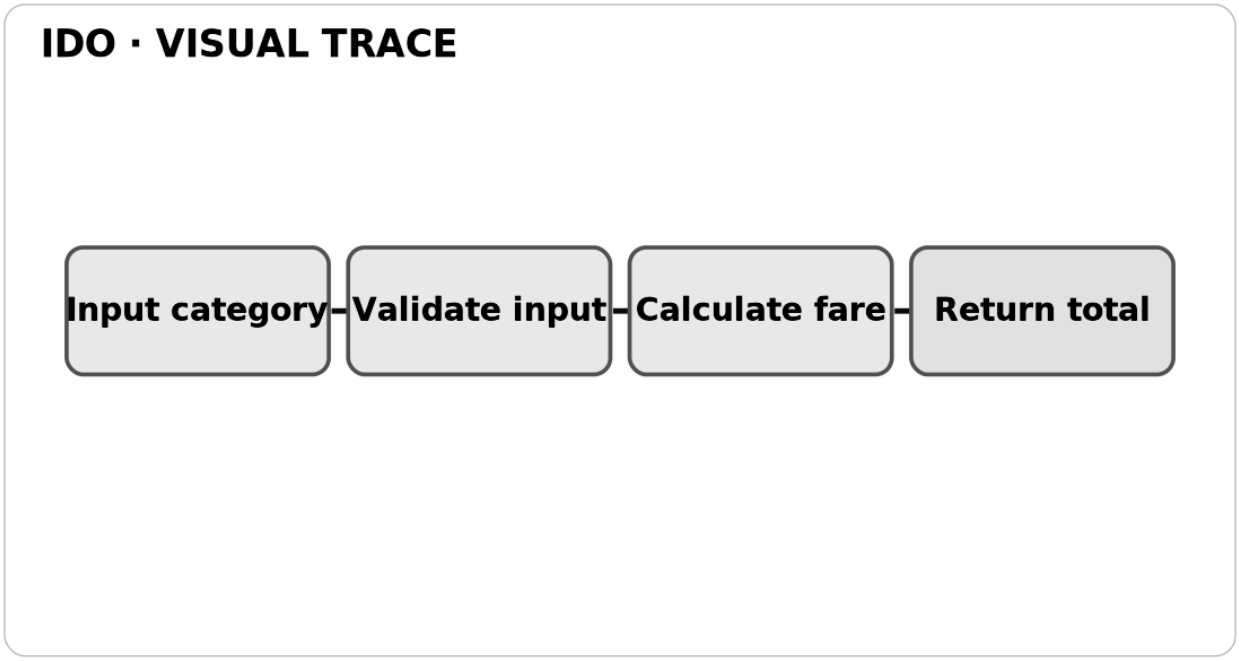
State the interface for each subtask; every step has a specific responsibility.

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Decompose a cinema price calculator for child/adult tickets.

State the interface for each subtask; every step has a specific responsibility.



IDO · VISUAL TRACE; Input category; Validate input; Calculate fare; Return total

WE DO — GUIDED PRACTICE

Try → Discuss → Improve

TRY: Attempt the guided questions in BLACK PEN. DISCUSS: Compare approaches with a partner, explain your reasoning and agree one improvement. IMPROVE: After the Exit Ticket, use the source CHECK model in GREEN PEN to correct and explain errors.

Guided 1 — TRY

Decompose a sports-club booking app into four named operations.
Source code / input:
ValidateMember → CheckPlaces → ReservePlace → SendConfirmation.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations or a justified explanation.

Question 1

Identify three independent subtasks and their outputs for a library fine system. [6]

Question 2

Write C# Fine(daysLate) for 20p/day capped at £2, zero for on-time returns. [6]

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations or a justified explanation.

Question 3

Explain two benefits and one limitation of decomposition when a team builds a school booking app. [6]

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking.

Splitting work means the components never need to communicate.

Circle: Confident / Getting there / Need help

Explain what you can now do independently.

GREEN PEN — Correct and explain

Choose one error. Write the correct method and explain precisely why your first response was wrong.

WE DO — CHECK AND IMPROVE

Source-grounded model answers

CHECK only after completing the guided practice and Exit Ticket. Use GREEN PEN to improve your method and explain any error.

Guided 1 — CHECK

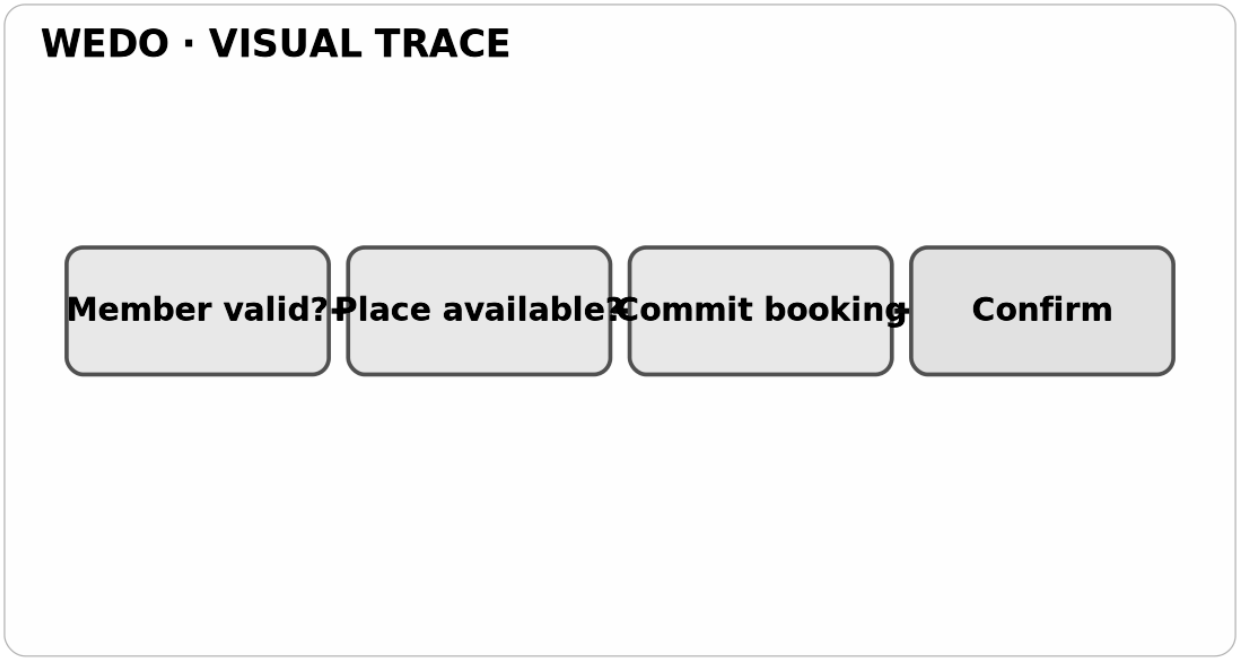
Follow the reasoning
Reservation must occur before confirming success; check for failures.

We Do source CHECK · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Decompose a sports-club booking app into four named operations.

Reservation must occur before confirming success; check for failures.



WEDO · VISUAL TRACE; Member valid?; Place available?; Commit booking; Confirm